

Intelligent Data Processor Core Foundation

Study Guide: Questions & Answers

This document summarizes the key concepts from the "Intelligent Data Processor Core Foundation" proposal, covering the design of an AI-driven ETL (Extract, Transform, Load) engine for processing partner data. It is organized into two sections, matching the source document, with review questions and answers for each.

Section I – Processing Engine Overview

This section explains how to design an AI engine that builds and manages ETL data pipelines for partner data, instead of relying on hand-coded scripts.

1. What are the three tiers of an AI-driven data processing engine's architecture?

The architecture has three tiers: (1) the Orchestration Layer, which schedules and manages task execution; (2) the Agentic / LLM Layer, the "engine" itself, which is an AI agent connected to data sources that can generate SQL or Python code; and (3) the Execution & Storage Layer, where data is actually processed and stored.

2. What role does the Orchestration Layer play, and which tools are given as examples?

The Orchestration Layer manages the execution of tasks using traditional schedulers or workflow managers. The examples given are Apache Airflow and Kestra (on Shakudo).

3. What is the function of the "Agentic / LLM Layer," and what does it use to generate SQL or Python code dynamically?

This layer is the core "engine" of the system. It consists of an AI agent connected to data sources and equipped with APIs that let it dynamically generate SQL or Python code, rather than relying on pre-written scripts.

4. Which platforms are mentioned as part of the Execution & Storage Layer?

Databricks, Snowflake, and PySpark are mentioned as the destinations and compute engines where data is physically moved and processed.

5. What are Dynamic Connection Tools used for in this system?

They allow the AI agent to list available databases, fetch table schemas, and safely authenticate API connections, essentially giving the agent the ability to explore and read data specifications on its own.

6. What example is given for a system prompt used to instruct the AI agent?

The example system prompt is: "You are a data engineering assistant. Given the source schema X and destination schema Y, write the SQL or PySpark code to clean the data and load it into the warehouse."

7. What is the purpose of Enrichment Functions, and what two AI functions are mentioned as examples?

Enrichment Functions use AI to parse messy, unstructured data (such as support tickets or reviews) before loading a structured summary into the database. The two example functions mentioned are `ai_classify` and `ai_extract`.

8. List the four steps in the step-by-step implementation of a basic AI ETL agent.

Step 1: Define the Source & Destination — set up API/JSON sources and database connections (e.g., PostgreSQL or Snowflake). Step 2: Initialize the Agent Tools — write Python functions that let the LLM read schemas and execute Python/SQL code. Step 3: Run the AI Agent — pass a user prompt to the LLM so it can request the schema, map columns, and write transformation logic. Step 4: Add Validation — integrate a schema validator or data quality checker before committing the transaction.

9. Why is a schema validator or data quality checker important in Step 4 (Add Validation)?

Because the AI generates its own transformation logic, a validation step is needed to make sure the LLM's output conforms to the database's constraints before the data is actually committed, preventing errors or corrupted data from reaching the destination.

10. Name three enterprise platforms mentioned that offer built-in AI-augmented ETL capabilities, and briefly describe what each contributes.

Databricks offers Lakeflow and Agentic AI functions (such as `ai_extract` and `ai_classify`) that embed AI models directly into ETL workflows. Matillion and Airbyte provide AI-assisted schema mapping and SQL optimization, reducing the manual effort of building pipelines. Informatica offers AI-powered mapping recommendations along with a conversational natural-language interface for generating pipelines.

11. What two clarifying questions does the document suggest asking before building a custom engine from scratch?

(1) What frameworks do we prefer to use, such as LangChain or LlamaIndex? (2) What are the primary data sources, such as APIs, local files, or relational databases?

Section II – Processing Engine Overview

This section covers the underlying tools and frameworks used to build or run the AI models themselves, either from scratch or by hosting existing open-source models locally.

12. What two general approaches are suggested for building a custom AI engine (in terms of frameworks)?

The document suggests either using a foundational machine learning framework (like PyTorch or TensorFlow) to build and train models, or using a self-hosted inference engine (like Ollama or vLLM) to run existing large language models locally on your own hardware.

13. What is the main benefit of using a self-hosted inference engine like Ollama or vLLM?

Running LLMs locally on your own hardware ensures data privacy and gives you total customization over the model and how it is used, rather than depending on a third-party API.

14. What distinguishes Ollama from vLLM in terms of use case?

Ollama is designed to be the easiest way to run and serve open-source LLMs locally (such as Llama 3 or Mistral) by bundling model weights and configuration into a single package. vLLM, on the other hand, is a high-throughput, memory-efficient serving engine optimized for production deployments that scale well on GPUs.

15. What makes LocalAI unique compared to Ollama and vLLM?

LocalAI is a self-hosted, drop-in replacement for the OpenAI API. It lets you run local LLMs, generate images, and transcribe audio on consumer-grade hardware, notably without requiring a GPU.

16. How do PyTorch and TensorFlow differ in their typical use cases?

PyTorch is a Python-based deep learning framework widely used for AI research, rapid prototyping, and dynamic model training. TensorFlow is an end-to-end platform developed by Google, better suited for numerical computation and deploying machine learning models across web, mobile, and edge devices.

17. What is Keras, and what frameworks can it run on top of?

Keras is a high-level neural networks API written in Python that is designed to be approachable for beginners. It can run on top of TensorFlow, PyTorch, or JAX.